

Lab Experiment 10

A* Search Algorithm

Aim

To write a Python program to implement the A* Search algorithm for finding the shortest path.

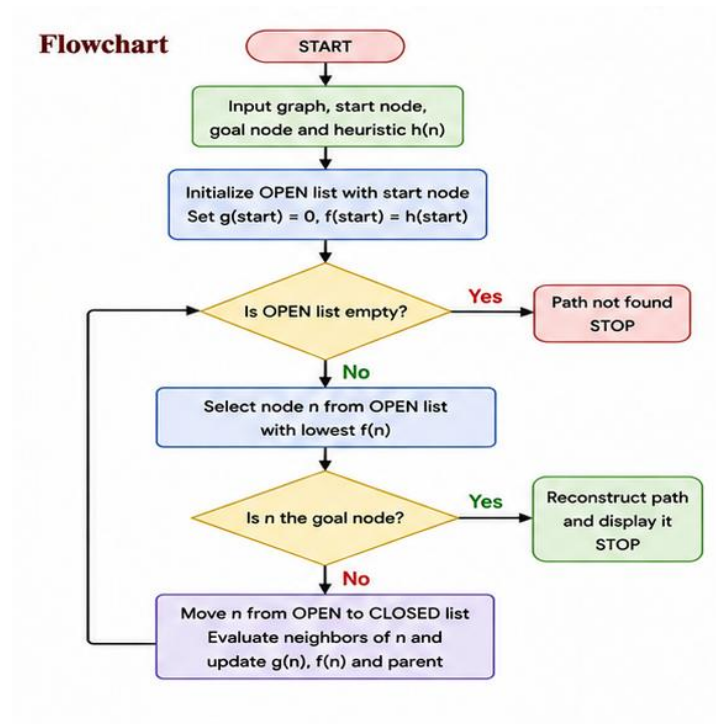
Objective

The objective of this experiment is to implement the A* Search algorithm using Python. It aims to find the shortest path between a start node and a goal node using a heuristic function.

Algorithm

1. Define the graph, start node, goal node, and heuristic values.
2. Add the start node to the priority queue.
3. Select the node with the lowest cost.
4. If the selected node is the goal, display the shortest path.
5. Otherwise, generate all neighboring nodes.
6. Calculate the total cost for each neighbor.
7. Repeat the process until the goal node is reached or no path exists.

Flowchart



Code

```
import heapq
```

```
graph = {  
    'A': [('B', 1), ('C', 4)],  
    'B': [('C', 2), ('D', 5)],  
    'C': [('D', 1)],  
    'D': []  
}
```

```
heuristic = {  
    'A': 4,  
    'B': 2,
```

```
'C': 1,  
'D': 0  
}
```

```
def a_star(start, goal):  
    queue = [(heuristic[start], 0, start, [])]  
    visited = set()  
  
    while queue:  
        f, g, node, path = heapq.heappop(queue)  
  
        if node in visited:  
            continue  
  
        path = path + [node]  
        visited.add(node)  
  
        if node == goal:  
            return path, g  
  
        for neighbor, cost in graph[node]:  
            if neighbor not in visited:  
                heapq.heappush(  
                    queue,  
                    (g + cost + heuristic[neighbor],  
                     g + cost,
```

```

        neighbor,
        path)
    )

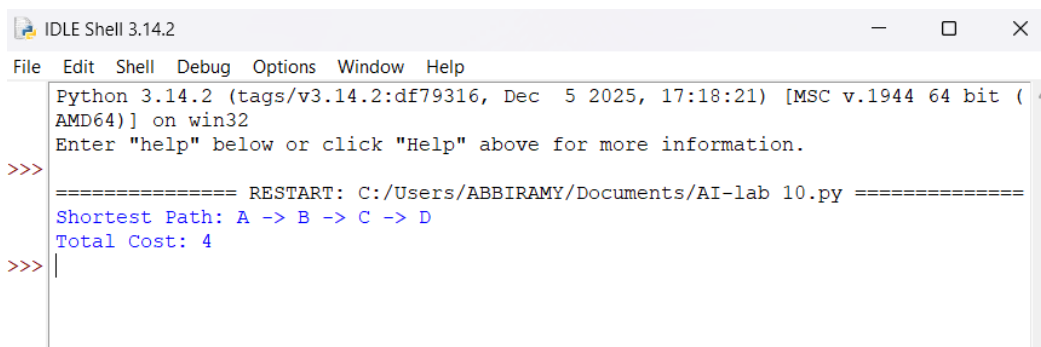
    return None

path, cost = a_star('A', 'D')

print("Shortest Path:", " -> ".join(path))
print("Total Cost:", cost)

```

Output



```

IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ABBIRAMY/Documents/AI-lab 10.py =====
Shortest Path: A -> B -> C -> D
Total Cost: 4
>>>

```

Result

The Python program successfully implemented the A* Search algorithm and found the shortest path between the start node and the goal node.

Conclusion

The A* Search algorithm efficiently finds the shortest path by combining the actual path cost and heuristic estimate. It is widely used in AI applications such as pathfinding and navigation.